Integrating Inventory with Wazuh

This guide will walk you through setting up and integrating the "Inventory" web panel with your existing Wazuh installation. Inventory is a simple web dashboard that pulls system details (like hardware, OS, network info, and more) from Wazuh's

1. Introduction

Wazuh Inventory is an open-source web dashboard that connects to your existing Wazuh SIEM server and pulls real-time system inventory data from all your monitored agents. It uses Wazuh's built-in SysCollector module to display hardware specs, operating systems, network interfaces, open ports, installed packages, and running processes all in one clean, searchable interface.

What makes Inventory useful is that it requires no extra software installed on your monitored machines. The Wazuh agents you already have running collect the data; the Inventory application simply reads that data through the Wazuh API and presents it on a web dashboard.

This guide walks you through every step needed to get the Inventory project running in your VirtualBox lab environment, from installing Python dependencies all the way to accessing the finished dashboard in your browser.

Component	OS / Role	Details
Wazuh Manager	Ubuntu Desktop	Central SIEM server — receives and analyzes all security events. Hosts the Wazuh Dashboard.
Wazuh Agent 1	Ubuntu Server (VirtualBox)	Agent machine where the Inventory project will be deployed. Already running the Wazuh agent.
Wazuh Agent 2	Windows Server (VirtualBox)	Second monitored agent. No additional setup required on this machine.
Inventory App	Ubuntu Server (same as Agent 1)	Python/Flask web dashboard that pulls

		hardware, OS, and network data from Wazuh's SysCollector API.

Important Notes Before Starting:

- This guide assumes you have a basic Linux setup (e.g., Ubuntu or similar) and are comfortable with terminal commands. If you're new to Linux, search for beginner tutorials on using the terminal.
- You must already have Wazuh installed and running, including the Wazuh server (SIEM) and agents deployed on the machines you want to monitor. If not, install Wazuh first from their official docs (wazuh.com).
- All commands are run in the terminal. Use sudo if needed for permissions.

- Prerequisites: Linux OS, Python 3.8 or higher (check with python3 --version), and access to your Wazuh server's IP, port, username, and password.
- The setup uses HTTPS for security, so we'll create a self-signed SSL certificate (fine for testing; use a real one for production).
- Default login after setup: Username = admin, Password = Meuadmin123. Change this immediately for security.

Follow these steps one by one. If something goes wrong, check the logs in the logs/ folder for errors.

Step 1 Open Terminal on Ubuntu Server

Log in to your Ubuntu Server virtual machine and open a terminal.

Step 2 Update the System and Install Python

First, make sure your system packages are up to date, then install Python 3 and pip if they are not already present.

Run these commands one by one:
sudo apt update

```
ubuntu@ubuntu:~$ sudo apt update
[sudo] password for ubuntu:
Get:1 https://download.docker.com/linux/ubuntu jammy InRelease [48.5 kB]
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 http://pk.archive.ubuntu.com/ubuntu jammy InRelease
Hit:4 https://dl.google.com/linux/chrome-stable/deb stable InRelease
Get:5 http://pk.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Hit:6 http://pk.archive.ubuntu.com/ubuntu jammy-backports InRelease
Ign:7 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages
Ign:8 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main i386 Packages
Get:7 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3,439 kB]
Err:7 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages
  File has unexpected size (3439716 != 3439088). Mirror sync in progress? [IP: 185.125.190.83 80]
  Hashes of expected file:
   - Filesize:3439088 [weak]
   - SHA256:ee472eb2838ec95592cc4535b480e1f1180b97ec76b8ce72a322eab19d692bb6
   - SHA1:b50006c4772c0144bff27c18c1004e4da8f317b6 [weak]
   - MD5Sum:d15b21e7df2f7c7dd2bd98ba59c0abe1 [weak]
  Release file created at: Thu, 30 Apr 2026 11:36:23 +0000
Get:8 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main i386 Packages [998 kB]
Err:8 http://pk.archive.ubuntu.com/ubuntu jammy-updates/main i386 Packages

Fetched 1,174 kB in 4s (297 kB/s)
Reading package lists... Done
N: Skipping acquisition of configured file 'main/binary-i386/Packages', as repository 'https://dl.google.com/linux/chrome-stable/deb stable InRelease' doesn't support architecture 'i386'
E: Failed to fetch http://pk.archive.ubuntu.com/ubuntu/dists/jammy-updates/main/binary-amd64/Packages.xz  File has unexpected size (3439716 != 3439088). Mirror sync in progress? [IP: 185.125.190.83
80]
   Hashes of expected file:
    - Filesize:3439088 [weak]
    - SHA256:ee472eb2838ec95592cc4535b480e1f1180b97ec76b8ce72a322eab19d692bb6
    - SHA1:b50006c4772c0144bff27c18c1004e4da8f317b6 [weak]
    - MD5Sum:d15b21e7df2f7c7dd2bd98ba59c0abe1 [weak]
   Release file created at: Thu, 30 Apr 2026 11:36:23 +0000
E: Failed to fetch http://pk.archive.ubuntu.com/ubuntu/dists/jammy-updates/main/binary-i386/Packages.xz
E: Some index files failed to download. They have been ignored, or old ones used instead.
ubuntu@ubuntu:~$
```

sudo apt install python3 python3-pip -y

python3 --version

```
ubuntu@ubuntu:~$ python3 --version
Python 3.10.12
ubuntu@ubuntu:~$
```

Step 3: Download the Wazuh Inventory Project

Clone the Inventory project repository from GitHub. This downloads all the project files to your machine.

git clone https://github.com/Maarckz/Inventory.git

```
ubuntu@ubuntu:~$ git clone https://github.com/Maarckz/Inventory.git
Cloning into 'Inventory'...
remote: Enumerating objects: 864, done.
remote: Counting objects: 100% (120/120), done.
remote: Compressing objects: 100% (65/65), done.
remote: Total 864 (delta 80), reused 54 (delta 54), pack-reused 744 (from 2)
Receiving objects: 100% (864/864), 22.67 MiB | 7.19 MiB/s, done.
Resolving deltas: 100% (481/481), done.
ubuntu@ubuntu:~$
```

cd Inventory

```
ubuntu@ubuntu:~$ ls
Desktop    Downloads                    emerging.rules.tar.gz  injector   main.py  mypasswords.txt  Pictures  rules      snap       test.exe       Videos
Documents  email_phishing_detector.py   index.html             Inventory  Music    nano.2594.save   Public    shell.php  Templates  venv_phishing  wazuh-agent_4.14.1-1_amd64.deb
ubuntu@ubuntu:~$ cd Inventory
ubuntu@ubuntu:~/Inventory$ ls
 app.py   Images                    LICENSE         models.py   postgres    requirements.txt   SECURITY.md   static      utils
 data     'Instalação INVENTORY.pdf'  migrate_db.py   NOTICE      README.md   reset_db.py        ssl           templates
ubuntu@ubuntu:~/Inventory$
```

You are now inside the project directory. All remaining commands in the setup should be run from inside this folder. You can confirm your location with:

Pwd

```
ubuntu@ubuntu:~/Inventory$ pwd
/home/ubuntu/Inventory
ubuntu@ubuntu:~/Inventory$
```

The output should show something like: /home/ubuntu/Inventory

Step 4 Set Up a Python Virtual Environment

A virtual environment keeps the project's Python packages isolated from the rest of the system. This avoids conflicts with other software and is the recommended approach, especially on Kali Linux or Ubuntu 22.04+ systems that follow PEP 668.

4.1 Install the Virtual Environment Tool

sudo apt install python3-venv python3-pip python3-dev -y

```
ubuntu@ubuntu:~/Inventory$ sudo apt install python3-venv python3-pip python3-dev -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-dev is already the newest version (3.10.6-1~22.04.1).
python3-pip is already the newest version (22.0.2+dfsg-1ubuntu0.7).
python3-venv is already the newest version (3.10.6-1~22.04.1).
0 to upgrade, 0 to newly install, 0 to remove and 49 not to upgrade.
ubuntu@ubuntu:~/Inventory$
```

4.2 Create the Virtual Environment

This creates a folder called 'venv-Inventory' inside your project directory that contains an isolated Python installation:

python3 -m venv venv-Inventory

```
ubuntu@ubuntu:~/Inventory$ python3 -m venv venv-Inventory
ubuntu@ubuntu:~/Inventory$
```

4.3 Activate the Virtual Environment

Activating the environment means all Python commands and package installations from this point will use the isolated environment, not the system Python:

source venv-Inventory/bin/activate

```
ubuntu@ubuntu:~/Inventory$ source venv-Inventory/bin/activate
(venv-Inventory) ubuntu@ubuntu:~/Inventory$
```

After activation, your terminal prompt will change to show (venv-Inventory) at the beginning, like this:
(venv-Inventory) ubuntu@ubuntu:~/Inventory$
NOTE: Every time you open a new terminal session and want to run the Inventory app, you must activate the virtual environment again with: source venv-Inventory/bin/activate

Step 5 Install Required Python Packages

Now install all the Python libraries the Inventory app needs to run. Make sure the virtual environment is active before running this command (your prompt should show (venv-Inventory)).

pip install flask bcrypt requests python-dotenv

```
ubuntu@ubuntu:~/Inventory$ source venv-Inventory/bin/activate
(venv-Inventory) ubuntu@ubuntu:~/Inventory$ pip install flask bcrypt requests python-dotenv
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 103.4/103.4 KB 976.1 kB/s eta 0:00:00
Collecting bcrypt
  Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl (278 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 278.2/278.2 KB 4.7 MB/s eta 0:00:00
Collecting requests
  Using cached requests-2.33.1-py3-none-any.whl (64 kB)
Collecting python-dotenv
  Downloading python_dotenv-1.2.2-py3-none-any.whl (22 kB)
Collecting markupsafe>=2.1.1
  Downloading markupsafe-3.0.3-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)
Collecting blinker>=1.9.0
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting werkzeug>=3.1.0
  Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 226.5/226.5 KB 14.5 MB/s eta 0:00:00
Collecting jinja2>=3.1.2
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 134.9/134.9 KB 11.7 MB/s eta 0:00:00
Collecting click>=8.1.3
  Downloading click-8.3.3-py3-none-any.whl (110 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 110.5/110.5 KB 14.4 MB/s eta 0:00:00
Collecting itsdangerous>=2.2.0
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting urllib3<3,>=1.26
  Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 131.6/131.6 KB 5.4 MB/s eta 0:00:00
Collecting idna<4,>=2.5
  Downloading idna-3.13-py3-none-any.whl (68 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 68.6/68.6 KB 8.7 MB/s eta 0:00:00
Collecting certifi>=2023.5.7
  Using cached certifi-2026.4.22-py3-none-any.whl (135 kB)
Collecting charset_normalizer<4,>=2
  Using cached charset_normalizer-3.4.7-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (216 kB)
Installing collected packages: urllib3, python-dotenv, markupsafe, itsdangerous, idna, click, charset_normalizer, certifi, blinker, bcrypt, werkzeug, requests, jinja2, flask
Successfully installed bcrypt-5.0.0 blinker-1.9.0 certifi-2026.4.22 charset_normalizer-3.4.7 click-8.3.3 flask-3.1.3 idna-3.13 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 python-dotenv-1.2.2 re
quests-2.33.1 urllib3-2.6.3 werkzeug-3.1.8
(venv-Inventory) ubuntu@ubuntu:~/Inventory$
```

Wait for the installation to complete. You will see download and installation progress messages. If you need the full set of packages including QR code and PDF report support, run:

pip3 install flask flask_session bcrypt requests python-dotenv qrcode pyotp reportlab

```
(venv-Inventory) ubuntu@ubuntu:~/Inventory$ pip3 install flask flask_session bcrypt requests python-dotenv qrcode pyotp reportlab
Requirement already satisfied: flask in ./venv-Inventory/lib/python3.10/site-packages (3.1.3)
Collecting flask_session
  Downloading flask_session-0.8.0-py3-none-any.whl (24 kB)
Requirement already satisfied: bcrypt in ./venv-Inventory/lib/python3.10/site-packages (5.0.0)
Requirement already satisfied: requests in ./venv-Inventory/lib/python3.10/site-packages (2.33.1)
Requirement already satisfied: python-dotenv in ./venv-Inventory/lib/python3.10/site-packages (1.2.2)
Collecting qrcode
  Downloading qrcode-8.2-py3-none-any.whl (45 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.0/46.0 KB 1.9 MB/s eta 0:00:00
Collecting pyotp
  Downloading pyotp-2.9.0-py3-none-any.whl (13 kB)
Collecting reportlab
  Downloading reportlab-4.5.0-py3-none-any.whl (2.0 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 5.1 MB/s eta 0:00:00
Requirement already satisfied: itsdangerous>=2.2.0 in ./venv-Inventory/lib/python3.10/site-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in ./venv-Inventory/lib/python3.10/site-packages (from flask) (3.1.6)
Requirement already satisfied: blinker>=1.9.0 in ./venv-Inventory/lib/python3.10/site-packages (from flask) (1.9.0)
Requirement already satisfied: markupsafe>=2.1.1 in ./venv-Inventory/lib/python3.10/site-packages (from flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in ./venv-Inventory/lib/python3.10/site-packages (from flask) (3.1.8)
Requirement already satisfied: click>=8.1.3 in ./venv-Inventory/lib/python3.10/site-packages (from flask) (8.3.3)
Collecting cachelib
  Downloading cachelib-0.13.0-py3-none-any.whl (20 kB)
Collecting msgspec>=0.18.6
  Downloading msgspec-0.21.1-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (222 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 222.7/222.7 KB 8.3 MB/s eta 0:00:00
Requirement already satisfied: idna<4,>=2.5 in ./venv-Inventory/lib/python3.10/site-packages (from requests) (3.13)
Requirement already satisfied: urllib3<3,>=1.26 in ./venv-Inventory/lib/python3.10/site-packages (from requests) (2.6.3)
Requirement already satisfied: certifi>=2023.5.7 in ./venv-Inventory/lib/python3.10/site-packages (from requests) (2026.4.22)
Requirement already satisfied: charset_normalizer<4,>=2 in ./venv-Inventory/lib/python3.10/site-packages (from requests) (3.4.7)
Collecting pillow>=9.0.0
  Downloading pillow-12.2.0-cp310-cp310-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (7.1 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.1/7.1 MB 10.2 MB/s eta 0:00:00
Installing collected packages: qrcode, pyotp, pillow, msgspec, cachelib, reportlab, flask_session
Successfully installed cachelib-0.13.0 flask_session-0.8.0 msgspec-0.21.1 pillow-12.2.0 pyotp-2.9.0 qrcode-8.2 reportlab-4.5.0
(venv-Inventory) ubuntu@ubuntu:~/Inventory$
```

NOTE: If you see an 'externally-managed-environment' error, it means you are trying to install packages outside a virtual environment. Go back to Step 4 and activate the venv first, then re-run the install command.

Step 6 Retrieve the Wazuh API Password

The Inventory app connects to your Wazuh Manager's API using the `wazuh-wui` user account. This step is performed on your Ubuntu Desktop (Wazuh Manager machine), not on Ubuntu Server.

6.1 Open Terminal on Ubuntu Desktop (Wazuh Manager)

Open a terminal on your Ubuntu Desktop machine and navigate to the Wazuh Dashboard configuration directory:

cd /usr/share/wazuh-dashboard/data/wazuh/config/

```
wazuh@fypserver:~$ cd /usr/share/wazuh-dashboard/data/wazuh/config/
-bash: cd: /usr/share/wazuh-dashboard/data/wazuh/config/: Permission denied
wazuh@fypserver:~$ sudo su
[sudo] password for wazuh:
root@fypserver:/home/wazuh# cd /usr/share/wazuh-dashboard/data/wazuh/config/
root@fypserver:/usr/share/wazuh-dashboard/data/wazuh/config#
```

Change Password:
cd /usr/share/wazuh-indexer/plugins/opensearch-security/tools/

sudo bash wazuh-passwords-tool.sh -A -u wazuh-wui -p "NewPass" -au wazuh -ap "Wui-Pass"

```
root@fypserver:/usr/share/wazuh-indexer/plugins/opensearch-security/tools# sudo bash wazuh-passwords-tool.sh -A -u wazuh-wui -p [redacted] -au wazuh -ap [redacted]
30/04/2026 21:57:08 INFO: The password for Wazuh API user wazuh-wui is [redacted]
30/04/2026 21:57:08 INFO: Updated wazuh-wui user password in wazuh dashboard. Remember to restart the service.
root@fypserver:/usr/share/wazuh-indexer/plugins/opensearch-security/tools#
```

Update Dashboard configuration

sudo sed -i 's/password: ".*"/password: "NewPass"/' /usr/share/wazuh-dashboard/data/wazuh/config/wazuh.yml

```
root@fypserver:/usr/share/wazuh-indexer/plugins/opensearch-security/tools# sudo sed -i 's/password: ".*"/password: [redacted]' /usr/share/wazuh-dashboard/data/wazuh/config/wazuh.yml
root@fypserver:/usr/share/wazuh-indexer/plugins/opensearch-security/tools#
```

sudo systemctl restart wazuh-dashboard

```
root@fypserver:/usr/share/wazuh-indexer/plugins/opensearch-security/tools# sudo systemctl restart wazuh-dashboard
root@fypserver:/usr/share/wazuh-indexer/plugins/opensearch-security/tools#
```

Update Inventory .env file

On your Ubuntu Server (where Inventory is installed):
cd /home/ubuntu/Inventory
sed -i 's/WAZUH_PASSWORD=.*/WAZUH_PASSWORD=NewPass/' .env

```
ubuntu@ubuntu:~$ cd Inventory
ubuntu@ubuntu:~/Inventory$ pwd
/home/ubuntu/Inventory
ubuntu@ubuntu:~/Inventory$ sed -i 's/WAZUH_PASSWORD=.*/WAZUH_PASSWORD=[redacted]/' .env
ubuntu@ubuntu:~/Inventory$
```

6.2 Open the Wazuh Configuration File On Manager

Open the wazuh.yml file to view the API credentials:

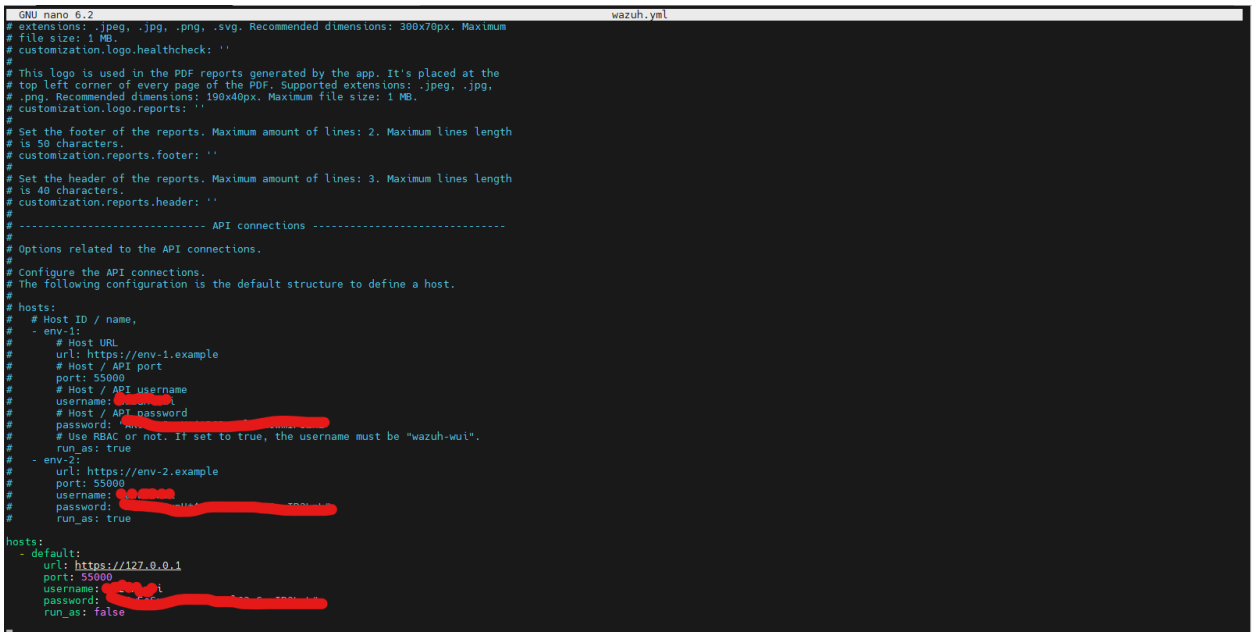
cd /usr/share/wazuh-dashboard/data/wazuh/config/
sudo nano wazuh.yml

```
root@fypserver:/usr/share/wazuh-dashboard/data/wazuh/config# nano wazuh.yml
root@fypserver:/usr/share/wazuh-dashboard/data/wazuh/config#
```

Scroll down to the hosts section near the bottom of the file. You will see a block that looks like this:

```
hosts:
  - default:
      url: https://127.0.0.1
      port: 55000
      username: wazuh-wui
      password: "Password"
      run_as: false
```

Copy the password value (the text between the quotes). You will need it in Step 7.



6.3 Change the URL from Loopback to Localhost

While the file is open, change the url line from https://127.0.0.1 to https://localhost. This is required for the API to be accessible:

```
url: https://localhost
```

Save the file and close it. Then restart the Wazuh Dashboard service to apply the change:

```
sudo systemctl restart wazuh-dashboard
```

```
root@fypserver:/usr/share/wazuh-dashboard/data/wazuh/config# systemctl restart wazuh-dashboard
root@fypserver:/usr/share/wazuh-dashboard/data/wazuh/config#
```

Step 7 Configure the Project (.env File)

Now go back to your Ubuntu Server terminal. Make sure you are inside the Inventory project folder and your virtual environment is active. You will now create the configuration file that tells the app how to connect to Wazuh and what settings to use.

nano .env

Paste the following configuration into the file. Replace the values marked with YOUR values:

```
# Security settings
SECRET_KEY=change_this_to_a_random_password
INVENTORY_DIR=data/inventory
AUTH_FILE=data/auth/logins.json

# Network settings
HOST=0.0.0.0
PORT=8000
DEBUG=False

# HTTPS settings
USE_HTTPS=True
SSL_CERT_PATH=ssl/cert.pem
SSL_KEY_PATH=ssl/key.pem

# Protection settings
MAX_LOGIN_ATTEMPTS=3
LOGIN_BLOCK_TIME=60
ALLOWED_IP_RANGES=10.10.90.0/24   # Change to your actual network range
BLOCKED_IPS_FILE=logs/blocked_ips.json

# Wazuh API configuration
WAZUH_PROTOCOL=https
WAZUH_HOST=10.10.90.141           # Replace with your Wazuh Manager IP
WAZUH_PORT=55000
WAZUH_USER=wazuh-wui
```

WAZUH_PASSWORD=NewPass* # Replace with your actual wazuh-wui password

```
  GNU nano 6.2                                                    .env *

##########################################
## CONFIGURAÇÕES DE SEGURANÇA E AMBIENTE ##
##########################################
SECRET_KEY=[redacted]
SESSION_SALT=supersecreta_altere_esta_chave_salt!
INVENTORY_DIR=data/inventory
GROUPS_DIR=data/groups
AUTH_FILE=data/auth/logins.json
LOG_DIR=logs

###########################
## CONFIGURAÇÕES DE REDE ##
###########################
HOST=0.0.0.0
PORT=8000
DEBUG=False

###########################
## CONFIGURAÇÕES DE HTTPS ##
###########################
USE_HTTPS=True
SSL_CERT_PATH=ssl/cert.pem
SSL_KEY_PATH=ssl/key.pem


MAX_LOGIN_ATTEMPTS=3
LOGIN_BLOCK_TIME=60
#####################################
## PERMITIR IPS DE FAIXA ESPECIFICA ##
#####################################
ALLOWED_IP_RANGES=[redacted]/24

WAZUH_PROTOCOL=https
WAZUH_HOST=[redacted]
WAZUH_PORT=55000
WAZUH_USER=[redacted]
WAZUH_PASSWORD=[redacted]

##############################
## BANCO DE DADOS PostgreSQL ##
##############################
DB_USER=inventorydb
DB_PASS=senhadoinventorydb
DB_HOST=localhost
DB_PORT=5432
DB_NAME=inventory_db
```

After pasting and editing the values, save and close the file:
NOTE: For ALLOWED_IP_RANGES, set it to the subnet of your VirtualBox network. For example, if your machines are on the 10.10.90.x range, use 10.10.90.0/24. If on 192.168.x.x, use 192.168.0.0/16. **Step 8 Generate SSL Certificates**

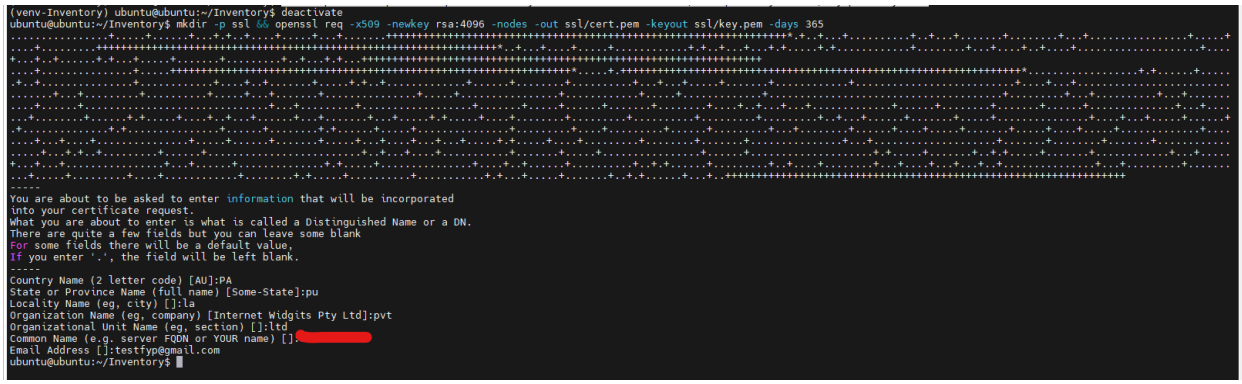
The Inventory app uses HTTPS for security, so it needs an SSL certificate and private key. For your lab environment, you will generate a self-signed certificate. Browsers will show a security warning for self-signed certificates, this is normal for internal testing.

Run this command to create both the ssl/ folder and the certificate files:

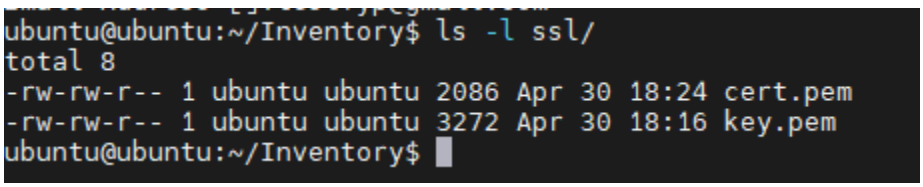
mkdir -p ssl && openssl req -x509 -newkey rsa:4096 -nodes -out ssl/cert.pem -keyout ssl/key.pem -days 365

OpenSSL will ask you several questions. For a test environment, you can press Enter to skip most fields. The only important one is Common Name type localhost:

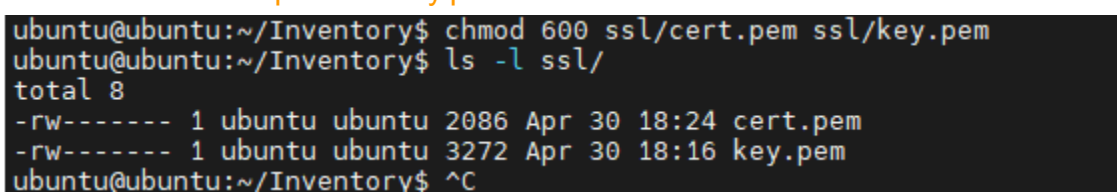
Country Name [AU]: .
State or Province Name [Some-State]: .
Locality Name []: .
Organization Name [Internet Widgits Pty Ltd]: .
Organizational Unit Name []: .
Common Name []: localhost
Email Address []: .



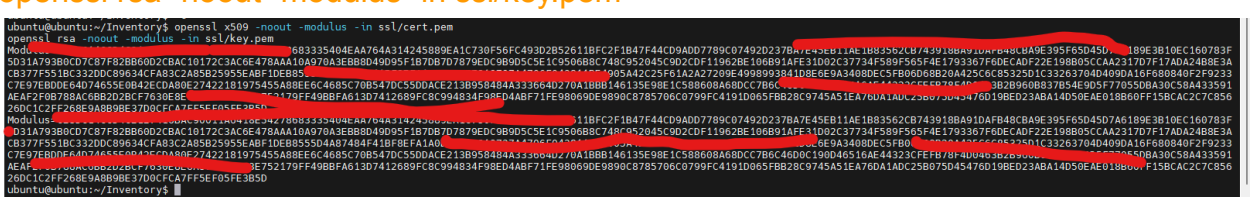
After the command finishes, verify that the certificate files were created:
ls -l ssl/



You should see two files: cert.pem and key.pem. If the permissions look too open, fix them:
chmod 600 ssl/cert.pem ssl/key.pem



To confirm the certificate and key match each other, run both of these and compare the Modulus output they must be identical:
openssl x509 -noout -modulus -in ssl/cert.pem
openssl rsa -noout -modulus -in ssl/key.pem



Step 9 Test the Data Collector

Before starting the web app, run the data collector script to fetch inventory data from your Wazuh Manager for the first time. This populates the `data/inventory/` folder with JSON files for each agent.

Navigate to the Project Directory

First, ensure you are in the correct working directory, which is the root of the `Inventory` project.

cd /home/ubuntu/Inventory

```
ubuntu@ubuntu:~/Inventory$ cd /home/ubuntu/Inventory
ubuntu@ubuntu:~/Inventory$ pwd
/home/ubuntu/Inventory
ubuntu@ubuntu:~/Inventory$
```

Download the Missing Script

Pip install -r requirements.txt

```
(venv-Inventory) ubuntu@ubuntu:~/Inventory$ pip install -r requirements.txt
Collecting Flask-APScheduler==1.13.1
  Downloading Flask-APScheduler-1.13.1.tar.gz (12 kB)
  Preparing metadata (setup.py) ... done
Collecting Flask-SQLAlchemy==3.1.1
  Downloading flask_sqlalchemy-3.1.1-py3-none-any.whl (25 kB)
Collecting psycopg2-binary==2.9.11
  Downloading psycopg2_binary-2.9.11-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (4.2 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.2/4.2 MB 3.4 MB/s eta 0:00:00
Requirement already satisfied: python-dotenv==1.2.2 in ./venv-Inventory/lib/python3.10/site-packages (from -r requirements.txt (line 4)) (1.2.2)
Requirement already satisfied: Flask-Session==0.8.0 in ./venv-Inventory/lib/python3.10/site-packages (from -r requirements.txt (line 5)) (0.8.0)
Collecting SQLAlchemy==2.0.48
  Downloading sqlalchemy-2.0.48-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (3.2 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 7.6 MB/s eta 0:00:00
Collecting reportlab==4.4.10
  Downloading reportlab-4.4.10-py3-none-any.whl (2.0 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 10.9 MB/s eta 0:00:00
Requirement already satisfied: requests==2.33.1 in ./venv-Inventory/lib/python3.10/site-packages (from -r requirements.txt (line 8)) (2.33.1)
Collecting asgiref==3.11.1
  Downloading asgiref-3.11.1-py3-none-any.whl (24 kB)
Requirement already satisfied: urllib3==2.6.3 in ./venv-Inventory/lib/python3.10/site-packages (from -r requirements.txt (line 10)) (2.6.3)
Requirement already satisfied: bcrypt==5.0.0 in ./venv-Inventory/lib/python3.10/site-packages (from -r requirements.txt (line 11)) (5.0.0)
Requirement already satisfied: Flask==3.1.3 in ./venv-Inventory/lib/python3.10/site-packages (from -r requirements.txt (line 12)) (3.1.3)
Requirement already satisfied: pyotp==2.9.0 in ./venv-Inventory/lib/python3.10/site-packages (from -r requirements.txt (line 13)) (2.9.0)
Requirement already satisfied: qrcode==8.2 in ./venv-Inventory/lib/python3.10/site-packages (from -r requirements.txt (line 14)) (8.2)
Collecting apscheduler<4.0.0,>=3.2.0
  Downloading apscheduler-3.11.2-py3-none-any.whl (64 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 64.4/64.4 KB 10.9 MB/s eta 0:00:00

Collecting python-dateutil>=2.4.2
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 229.9/229.9 KB 14.5 MB/s eta 0:00:00
Requirement already satisfied: cachelib in ./venv-Inventory/lib/python3.10/site-packages (from Flask-Session==0.8.0->-r requirements.txt (line 5)) (0.13.0)
Requirement already satisfied: msgspec>=0.18.6 in ./venv-Inventory/lib/python3.10/site-packages (from Flask-Session==0.8.0->-r requirements.txt (line 5)) (0.21.1)
Collecting typing-extensions>=4.6.0
  Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.6/44.6 KB 3.4 MB/s eta 0:00:00

Collecting greenlet>=1
  Downloading greenlet-3.5.0-cp310-cp310-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (613 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 613.4/613.4 KB 13.1 MB/s eta 0:00:00
Requirement already satisfied: charset-normalizer in ./venv-Inventory/lib/python3.10/site-packages (from reportlab==4.4.10->-r requirements.txt (line 7)) (3.4.7)
Requirement already satisfied: pillow>=9.0.0 in ./venv-Inventory/lib/python3.10/site-packages (from reportlab==4.4.10->-r requirements.txt (line 7)) (12.2.0)
Requirement already satisfied: certifi>=2023.5.7 in ./venv-Inventory/lib/python3.10/site-packages (from requests==2.33.1->-r requirements.txt (line 8)) (2026.4.22)
Requirement already satisfied: idna<4,>=2.5 in ./venv-Inventory/lib/python3.10/site-packages (from requests==2.33.1->-r requirements.txt (line 8)) (3.13)
Requirement already satisfied: click>=8.1.3 in ./venv-Inventory/lib/python3.10/site-packages (from Flask==3.1.3->-r requirements.txt (line 12)) (8.3.3)
Requirement already satisfied: werkzeug>=3.1.0 in ./venv-Inventory/lib/python3.10/site-packages (from Flask==3.1.3->-r requirements.txt (line 12)) (3.1.8)
Requirement already satisfied: jinja2>=3.1.2 in ./venv-Inventory/lib/python3.10/site-packages (from Flask==3.1.3->-r requirements.txt (line 12)) (3.1.6)
Requirement already satisfied: itsdangerous>=2.2.0 in ./venv-Inventory/lib/python3.10/site-packages (from Flask==3.1.3->-r requirements.txt (line 12)) (2.2.0)
Requirement already satisfied: blinker>=1.9.0 in ./venv-Inventory/lib/python3.10/site-packages (from Flask==3.1.3->-r requirements.txt (line 12)) (1.9.0)
Requirement already satisfied: markupsafe>=2.1.1 in ./venv-Inventory/lib/python3.10/site-packages (from Flask==3.1.3->-r requirements.txt (line 12)) (3.0.3)
Collecting tzlocal>=3.0
  Downloading tzlocal-5.3.1-py3-none-any.whl (18 kB)
Collecting six>=1.5
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Using legacy 'setup.py install' for Flask-APScheduler, since package 'wheel' is not installed.
Installing collected packages: tzlocal, typing-extensions, six, reportlab, psycopg2-binary, greenlet, SQLAlchemy, python-dateutil, asgiref, apscheduler, Flask-SQLAlchemy, Flask-APScheduler
  Attempting uninstall: reportlab
    Found existing installation: reportlab 4.5.0
```

Install the Plugin: Run the following command to install the `docker-compose-plugin` package:

sudo apt install docker-compose-plugin -y

```
root@ubuntu:/home/ubuntu# sudo apt install docker-compose-plugin -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker-compose-plugin is already the newest version (5.1.3-1~ubuntu.22.04~jammy).
0 upgraded, 0 newly installed, 0 to remove and 49 not upgraded.
root@ubuntu:/home/ubuntu#
```

cd /home/ubuntu/Inventory/postgres
sudo docker compose up -d

```
ubuntu@ubuntu:~/Inventory$ cd postgres
ubuntu@ubuntu:~/Inventory/postgres$ ls
docker-compose.yml
ubuntu@ubuntu:~/Inventory/postgres$ docker-compose up -d
Command 'docker-compose' not found, but can be installed with:
sudo snap install docker          # version 29.3.1, or
sudo apt  install docker-compose  # version 1.29.2-1
See 'snap info docker' for additional versions.
ubuntu@ubuntu:~/Inventory/postgres$ ^C
ubuntu@ubuntu:~/Inventory/postgres$ sudo docker compose up -d
[sudo] password for ubuntu:
[+] up 19/19
 ✔ Image postgres:16              Pulled
 ✔ Network postgres_default       Created
 ✔ Volume postgres_postgres_data  Created
 ✔ Container inventory_postgres   Started
ubuntu@ubuntu:~/Inventory/postgres$
```

For stoping:

cd /home/ubuntu/Inventory/postgres

sudo docker compose down

```
root@ubuntu:/home/ubuntu# cd /home/ubuntu/Inventory/postgres
sudo docker compose down
[+] down 1/1
 ✔ Network postgres_default Removed
root@ubuntu:/home/ubuntu/Inventory/postgres#
```

Once you have a PostgreSQL container running (using either of the methods above):

1. Activate the Virtual Environment: This step is essential every time you work on the project.

cd /home/ubuntu/Inventory

source venv-Inventory/bin/activate

```
ubuntu@ubuntu:~/Inventory$ cd /home/ubuntu/Inventory
source venv-Inventory/bin/activate
(venv-Inventory) ubuntu@ubuntu:~/Inventory$
```

2. Run the Database Migration: This creates all the necessary tables in your PostgreSQL database.

python3 migrate_db.py

```
(venv-Inventory) ubuntu@ubuntu:~/Inventory$ python3 migrate_db.py
Iniciando migração...
Usuários migrados com sucesso.
Hosts migrados com sucesso.
Grupos migrados com sucesso.
Migração concluída com sucesso.
(venv-Inventory) ubuntu@ubuntu:~/Inventory$
```

3. Run the Collector: This script will fetch the data from your Wazuh manager.

python3 utils/collector.py

```
ubuntu@ubuntu:~/Inventory$ cd /home/ubuntu/Inventory
source venv-Inventory/bin/activate
python3 utils/collector.py
DEBUG:tzlocal:/etc/timezone found, contents:
 Asia/Karachi

DEBUG:tzlocal:/etc/localtime found
DEBUG:tzlocal:2 found:
 {'/etc/timezone': 'Asia/Karachi', '/etc/localtime is a symlink to': 'Asia/Karachi'}
INFO:apscheduler.scheduler:Scheduler started
DEBUG:apscheduler.scheduler:Looking for jobs to run
DEBUG:apscheduler.scheduler:No jobs; waiting until a job is added
INFO:apscheduler.scheduler:Added job "wazuh_sync" to job store "default"
INFO:app:Sincronização agendada para cada 3600 segundos.
DEBUG:apscheduler.scheduler:Looking for jobs to run
DEBUG:apscheduler.scheduler:Next wakeup is due at 2026-04-30 23:58:25.664773+05:00 (in 3599.997955 seconds)
INFO:app:[Coletor] Iniciando ciclo de sincronização robusta...
DEBUG:urllib3.connectionpool:Starting new HTTPS connection (1):[redacted]:55000
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /security/user/authenticate?raw=true HTTP/1.1" 200 404
INFO:app:[Coletor] Autenticação bem-sucedida.
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /groups?pretty=true HTTP/1.1" 200 420
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /agents?select=id,name,ip,lastKeepAlive,status,os.platform,os.name,os.version,group&limit=1000 HTTP/1.1" 200 865
INFO:app:[Coletor] API retornou 3 agentes.
DEBUG:urllib3.connectionpool:Starting new HTTPS connection (2):[redacted]:55000
DEBUG:urllib3.connectionpool:Starting new HTTPS connection (3):[redacted]55000
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/001/hardware HTTP/1.1" 200 420
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/000/hardware HTTP/1.1" 200 446
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/002/hardware HTTP/1.1" 200 420
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/001/os HTTP/1.1" 200 512
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/000/os HTTP/1.1" 200 581
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/002/os HTTP/1.1" 200 578
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/000/packages HTTP/1.1" 200 219852
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/001/packages HTTP/1.1" 200 8408
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/002/packages HTTP/1.1" 200 219722
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/001/ports HTTP/1.1" 200 18743
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/000/ports HTTP/1.1" 200 22756
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/002/ports HTTP/1.1" 200 8232
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/001/processes HTTP/1.1" 200 34744
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/000/processes HTTP/1.1" 200 208881
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/002/processes HTTP/1.1" 200 140255
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/001/netaddr HTTP/1.1" 200 823
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/000/netaddr HTTP/1.1" 200 491
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/001/netiface HTTP/1.1" 200 904
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/002/netaddr HTTP/1.1" 200 1287
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/000/netiface HTTP/1.1" 200 801
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/001/netproto HTTP/1.1" 200 686
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/002/netiface HTTP/1.1" 200 1723
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/000/netproto HTTP/1.1" 200 418
DEBUG:urllib3.connectionpool:https://10.10.80.56:55000 "GET /syscollector/002/netproto HTTP/1.1" 200 992
INFO:app:[Coletor] Ciclo finalizado. 3 sincronizados. 0 movidos para legado.
INFO:app:[Coletor] Cache do Dashboard invalidado para atualização imediata.
(venv-Inventory) ubuntu@ubuntu:~/Inventory$
```

4.Start the Web App:

python3 app.py

```
(venv-Inventory) ubuntu@ubuntu:~/Inventory$ python3 -m utils.collector
(venv-Inventory) ubuntu@ubuntu:~/Inventory$ python3 app.py
 * IPs do servidor: ['localhost', '10.10.80.68', '127.0.0.1', '172.18.0.1']
 * Redes permitidas: ['10.10.80.0/24']
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on https://127.0.0.1:8000
 * Running on https://10.10.80.68:8000
Press CTRL+C to quit
^C(venv-Inventory) ubuntu@ubuntu:~/Inventory$
```

NOTE: If you see an authentication error, double-check the WAZUH_HOST and WAZUH_PASSWORD values in your .env file. Also confirm the Wazuh API is reachable with:

curl -k https://<WAZUH_HOST>:55000

```
(venv-Inventory) ubuntu@ubuntu:~/Inventory$ curl -k https://10.10.80.56:55000
{"title": "Unauthorized", "detail": "No authorization token provided"}(venv-Inventory) ubuntu@ubuntu:~/Inventory$ ^C
(venv-Inventory) ubuntu@ubuntu:~/Inventory$
```

Step 10 Set Up Automatic Data Refresh (Cron Job)

To keep the inventory data fresh, set up a cron job that runs the data collector script every 5 minutes automatically in the background.

Open the crontab editor:

sudo su

crontab -e

```
root@ubuntu:/home/ubuntu# crontab -e
crontab: installing new crontab
root@ubuntu:/home/ubuntu#
```

Then add the following line at the end of the file. Replace the path with your actual project path if it is different:

0 0 * * * cd /home/ubuntu/Inventory && /home/ubuntu/Inventory/venv-Inventory/bin/python3 /home/ubuntu/Inventory/utils/collector.py >> /home/ubuntu/Inventory/logs/collector_cron.log 2>&1

```
  GNU nano 6.2                                                    /tmp/crontab.71yuYi/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
*/5 * * * * /usr/local/bin/coleta_historico_chrome.sh
0 0 * * * cd /home/ubuntu/Inventory && /home/ubuntu/Inventory/venv-Inventory/bin/python3 /home/ubuntu/Inventory/utils/collector.py >> /home/ubuntu/Inventory/logs/collector_cron.log 2>&1
```

Save and close the file (Ctrl+X, then Y, then Enter). Cron will automatically pick up the job. The 0 0 means it runs every midnight daily . You do not need to restart anything cron handles it.

Log Rotation Configuration (Disk Space Control)

sudo nano /etc/logrotate.d/inventory

```
root@ubuntu:/home/ubuntu# sudo nano /etc/logrotate.d/inventory
root@ubuntu:/home/ubuntu#
```

```
/home/ubuntu/Inventory/logs/collector_cron.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 644 ubuntu ubuntu
}
```

```
  GNU nano 6.2                                                    /etc/logrotate.d/inventory
/home/ubuntu/Inventory/logs/collector_cron.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 644 ubuntu ubuntu
}
```

Step 11 Start the Web Application

Now start the Inventory web server. Make sure your virtual environment is still active (you should see (venv-Inventory) in your prompt). Run the following command from inside the Inventory folder:

python3 app.py

```
(venv-Inventory) ubuntu@ubuntu:~/Inventory$ python3 app.py
 * IPs do servidor: [[redacted], 'localhost', '127.0.0.1', '10.10.80.68']
 * Redes permitidas: [[redacted]4']
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on https://127.0.0.1:8000
 * Running on https:[redacted]
Press CTRL+C to quit
[redacted] - - [30/Apr/2026 20:46:04] "GET / HTTP/1.1" 302 -
[redacted] - - [30/Apr/2026 20:46:04] "GET /login HTTP/1.1" 200 -
[redacted] - - [30/Apr/2026 20:46:04] "GET /static/logo.svg HTTP/1.1" 200 -
[redacted] - - [30/Apr/2026 20:46:04] "GET /static/favicon.ico HTTP/1.1" 200 -
```

If the app starts successfully, you will see output similar to this:
[INFO] in app: IPs do servidor: ['10.10.70.60', '127.0.0.1', 'localhost']
[INFO] in app: Redes permitidas: ['10.10.70.0/24']
 * Serving Flask app 'app'
 * Debug mode: off
 * Running on all addresses (0.0.0.0)
 * Running on https://127.0.0.1:8000
 * Running on https://10.10.70.141:8000

The app is now running. Leave this terminal open. Do not close it closing it will stop the server.

NOTE: If you see a warning that says 'This is a development server. Do not use it in a production deployment', that is expected. For your lab environment, this is fine. For a production server, you would use a production WSGI server like gunicorn.

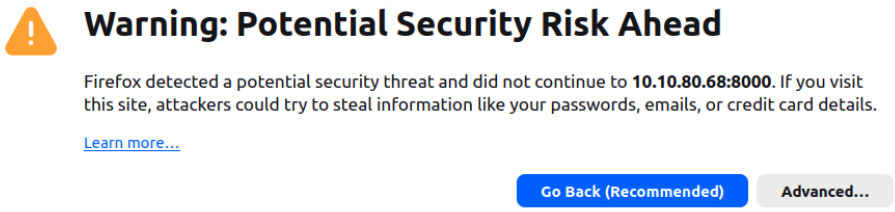
Step 12 Access the Inventory Dashboard

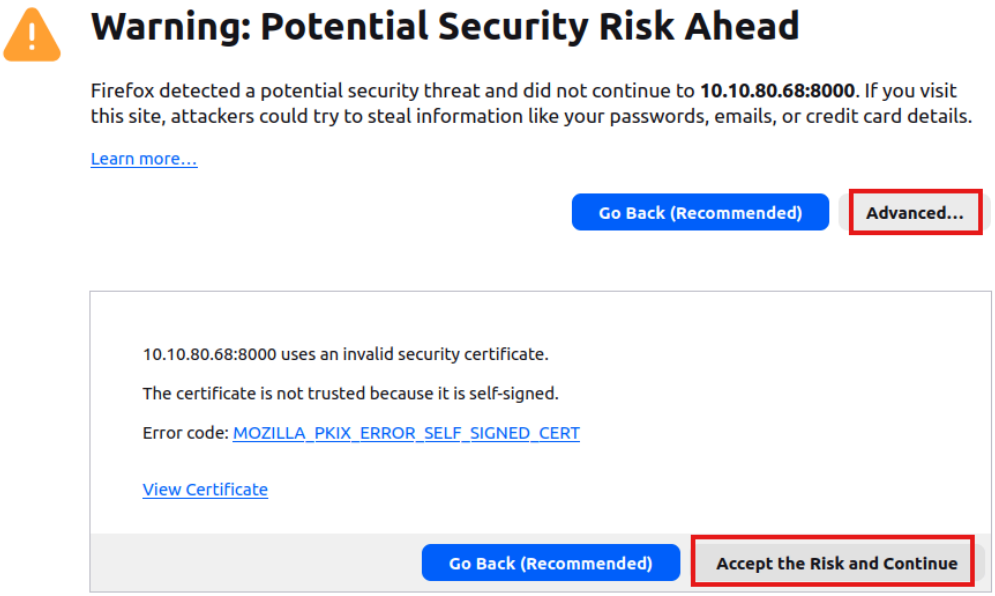
Open a web browser on any machine in your VirtualBox network (your Ubuntu Desktop, Ubuntu Server, or Windows Server).

12.1 Open the Dashboard

Navigate to the following address in your browser. Replace the IP with your Ubuntu Server's IP address:

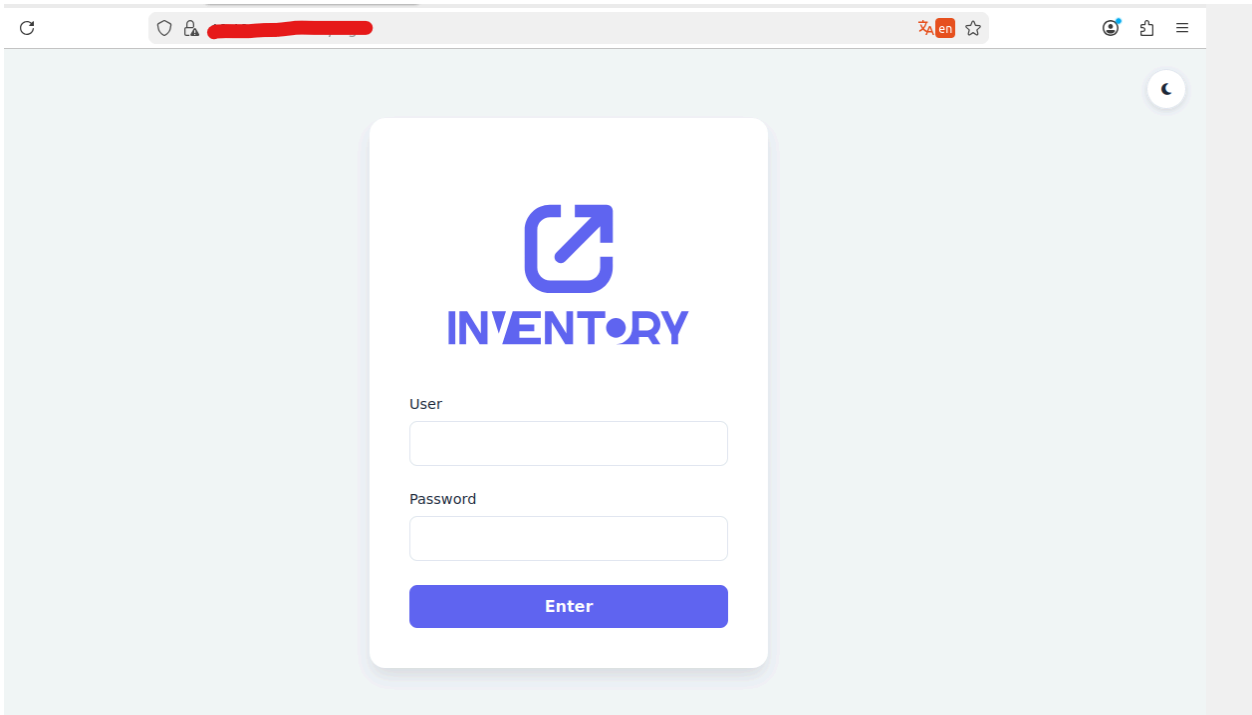
https://Ubuntu-Server-IP:8000





Your browser will show a security warning because you are using a self-signed SSL certificate. This is expected. Click 'Advanced' and then 'Proceed' (or 'Accept the Risk and Continue' in Firefox) to continue.

12.2 Log In



Use the default credentials to log in:

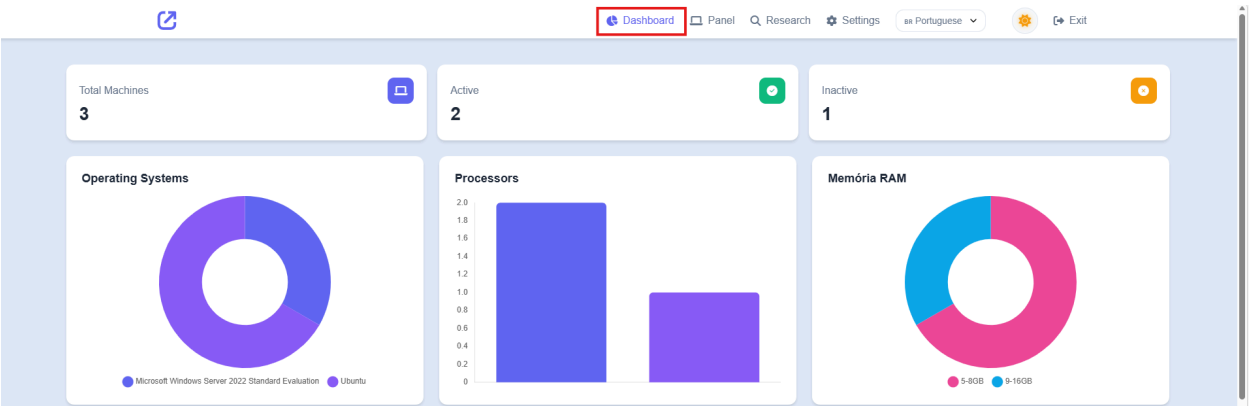
Username: admin
Password: Meuadmin123

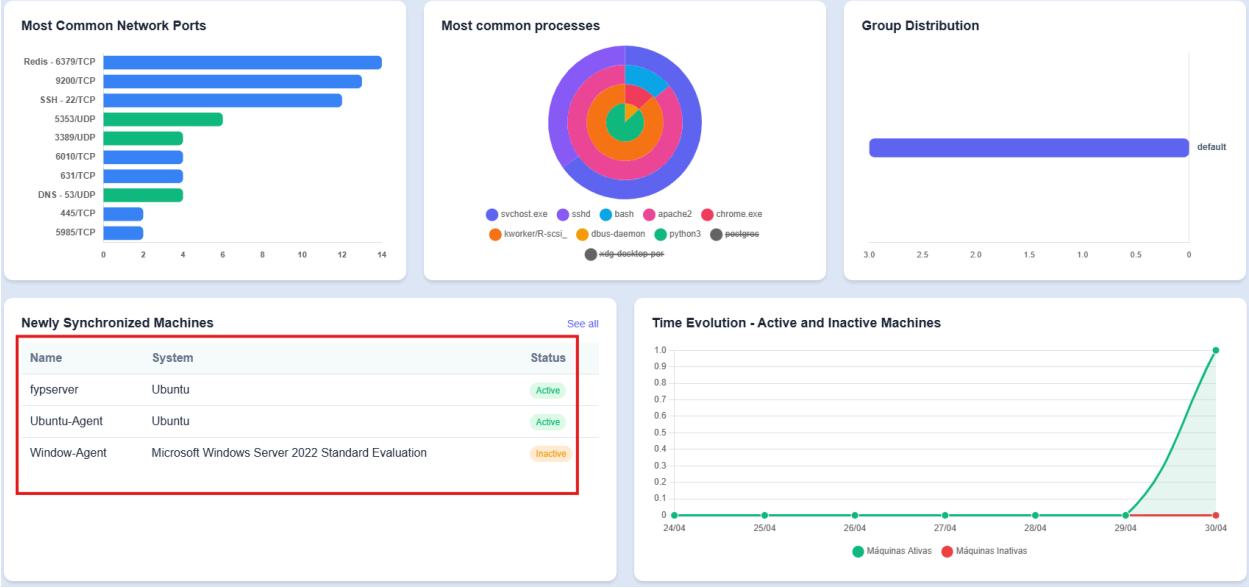
12.3 What You Should See

After logging in, the Dashboard page shows:

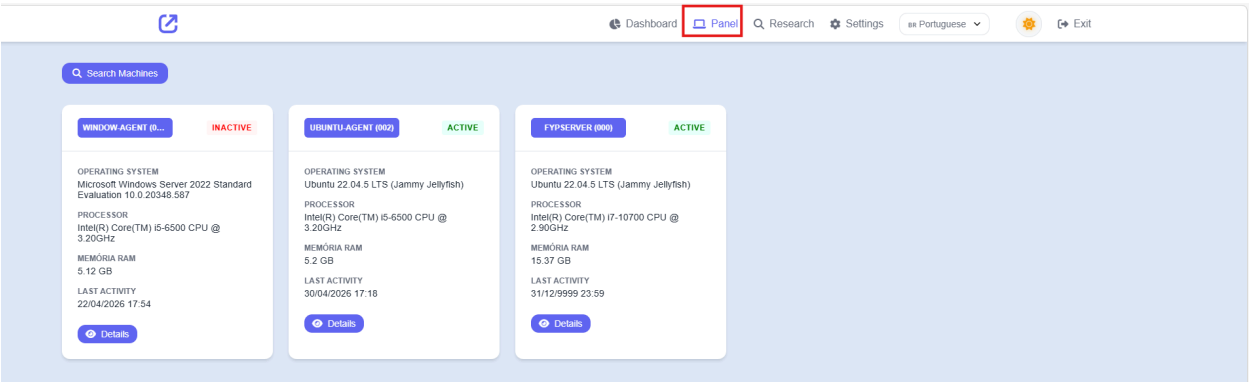
- Total Machines: number of agents Wazuh is monitoring
- Active Assuals: agents currently online and communicating
- Inactive: agents that are offline or disconnected
- Charts showing OS distribution, processor types, and memory RAM distribution across all agents

Dashboard:



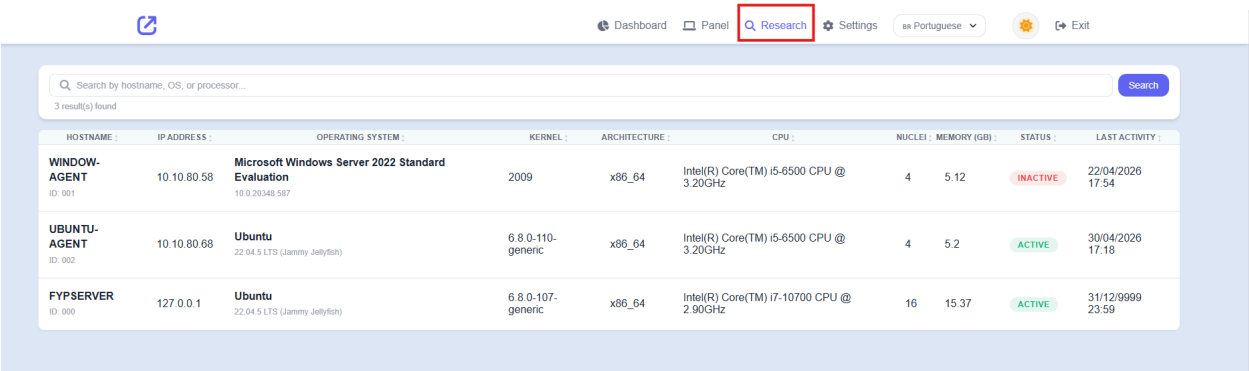


Panel:

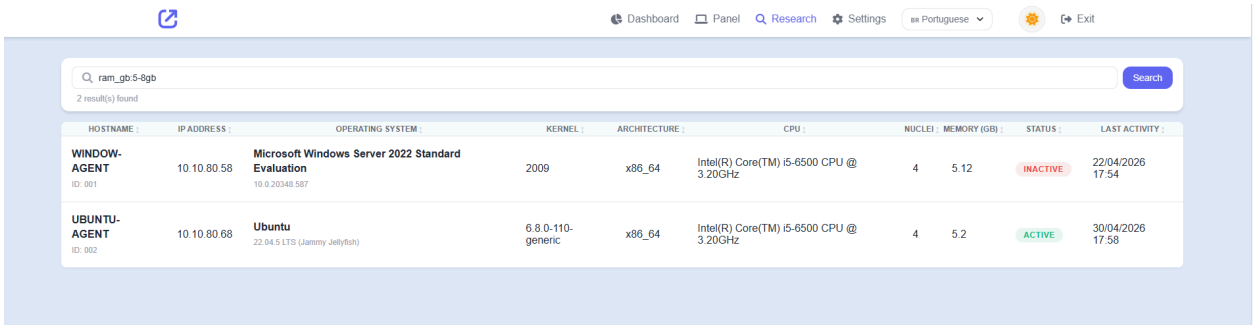


UBUNTU-AGENT
ACTIVE
Basic Information
Hostname
UBUNTU-AGENT
Agent ID
002
Operating System
Ubuntu 22.04.5 LTS (Jammy Jellyfish)
Architecture
x86_64
Motherboard
0
Groups
default
Last Activity
30/04/2026 17:58
Hardware
Processor
Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz
Colors
4
RAM Memory
5.2 GB

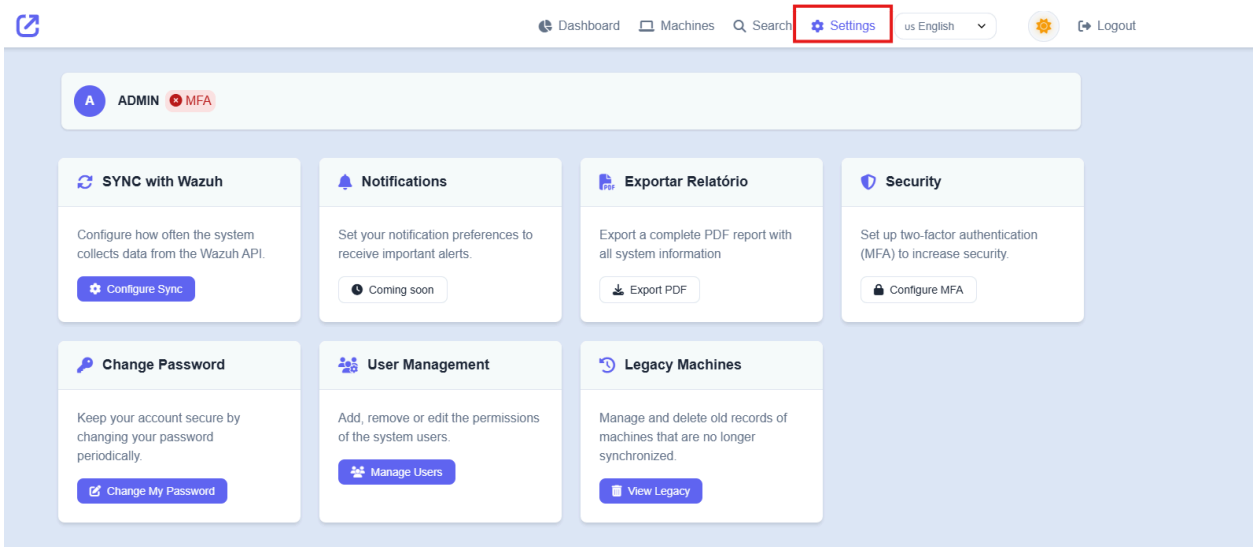
Research:


Dashboard
Panel
Research
Settings
Portuguese
Exit
Search by hostname, OS, or processor...
Search
3 result(s) found
HOSTNAME
IP ADDRESS
OPERATING SYSTEM
KERNEL
ARCHITECTURE
CPU
NUCLEI
MEMORY (GB)
STATUS
LAST ACTIVITY
WINDOW-AGENT
ID: 001
10.10.80.58
Microsoft Windows Server 2022 Standard Evaluation
10.0.20348.587
2009
x86_64
Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz
4
5.12
INACTIVE
22/04/2026 17:54
UBUNTU-AGENT
ID: 002
10.10.80.68
Ubuntu
22.04.5 LTS (Jammy Jellyfish)
6.8.0-110-generic
x86_64
Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz
4
5.2
ACTIVE
30/04/2026 17:18
FYPSERVER
ID: 000
127.0.0.1
Ubuntu
22.04.5 LTS (Jammy Jellyfish)
6.8.0-107-generic
x86_64
Intel(R) Core(TM) i7-10700 CPU @ 2.90GHz
16
15.37
ACTIVE
31/12/9999 23:59

inventory:hardware:intel(r) core(tm) i5-6500 cpu @ 3.20ghz
Pesquisar
2 resultado(s) encontrado(s)
HOSTNAME
ENDEREÇO IP
SISTEMA OPERACIONAL
KERNEL
ARQUITETURA
CPU
NÚCLEOS
MEMÓRIA (GB)
STATUS
ÚLTIMA ATIVIDADE
WINDOW-AGENT
ID: 001
10.10.80.58
Microsoft Windows Server 2022 Standard Evaluation
10.0.20348.587
2009
x86_64
Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz
4
5.12
INATIVO
22/04/2026 17:54
UBUNTU-AGENT
ID: 002
10.10.80.68
Ubuntu
22.04.5 LTS (Jammy Jellyfish)
6.8.0-110-generic
x86_64
Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz
4
5.2
ATIVO
30/04/2026 17:58



Settings:



The Panel page shows cards for each individual agent with OS type, processor, memory, and last activity. Clicking Details on any agent shows its full hardware, network interfaces, open ports, network settings, installed packages, and running processes.

The Research page lets you search across all agents by hostname, OS, or processor to quickly find specific machines.

Summary: In this task, I implemented a custom inventory dashboard that integrates with Wazuh SIEM to automatically collect and display system inventory data from all monitored agents in a controlled lab environment. The main objective was to fetch real-time hardware, operating system, installed packages, open ports, running processes, and network interface details from Wazuh agents via the Wazuh API, and present them in a clean, searchable web dashboard without manual data collection.

First, I set up the Inventory application on my Ubuntu Server where the Wazuh Agent is already installed. I cloned the Inventory project from GitHub, created a Python virtual environment, and installed all required dependencies including Flask, requests, python-dotenv, bcrypt, and PostgreSQL drivers. I also installed Docker Compose to run a PostgreSQL container because the application uses a database instead of JSON files for better scalability.

The Wazuh Agent collects inventory data using its built-in Syscollector module, which is enabled by default. To allow the Inventory app to fetch this data, I retrieved the `wazuh-wui` API user credentials from the Wazuh Manager's configuration file (`wazuh.yml`) and stored them in the `.env` file of the Inventory project. I also configured the application to use HTTPS by generating a self-signed SSL certificate and set the allowed IP range to match my VirtualBox network.

On the Ubuntu Server, I started the PostgreSQL container using `docker compose up -d`, ran the database migration script (`migrate_db.py`), and executed the data collector (`utils/collector.py`). Initially, the collector script failed to run because it lacked a main execution block and a proper module path. I fixed this by adding `sys.path.insert(0, ...)` at the top of the collector script to include the project root, and I also ensured the virtual environment was always activated. After these fixes, the collector successfully authenticated with the Wazuh API and fetched inventory data for all three agents (fypserver, Window-Agent, and Ubuntu-Agent).

I verified the database population by counting records in the `HostInventory` table, which showed 3 hosts. I then started the Flask web application (`app.py`) and accessed the dashboard at `https://<server_ip>:8000`. The dashboard displayed total machines, active/inactive counts, OS distribution charts, and individual agent cards. Clicking on any agent revealed detailed information including CPU, RAM, network interfaces, open ports, installed packages, and running processes – all sourced directly from Wazuh's Syscollector module.

To automate data refreshes, I scheduled a cron job to run the collector script every night at midnight, with log rotation configured to manage disk space. The dashboard also has a built-in scheduler that runs every hour, ensuring inventory stays up-to-date.

This implementation demonstrates how Wazuh's Syscollector data can be visualized through a custom dashboard without expensive commercial tools. It provides centralized visibility into all endpoints, helps with asset management, and is ideal for lab-based testing or for SOC teams to quickly audit system configurations and detect unauthorized changes.